

이동형

DevOps Engineer | 경력기술서

핵심 역량 (Core Competencies)

클라우드 인프라 설계 및 운영

AWS, GCP 기반 멀티·하이브리드 클라우드 아키텍처 설계 및 운영 경험

- AWS EKS, GCP GKE 프로덕션 환경 운영 및 비용 최적화 (하이브리드 전환 20% 절감, GCP 마이그레이션 50% 절감, 절감분 신규 프로젝트 재투자)
- Terraform을 활용한 IaC 기반 인프라 자동화 및 버전 관리 (GCP 리소스 100% 코드화)
- 온프레미스-클라우드 하이브리드 아키텍처 설계 및 마이그레이션 경험

CI/CD 파이프라인 구축 및 GitOps

GitOps 기반 배포 자동화로 개발 생산성 극대화

- GitHub Actions, GitLab CI, ArgoCD를 활용한 CI/CD 파이프라인 설계 및 고도화
- 배포 시간 50~67% 단축 및 주간 배포 3배 증대로 Time to Market 단축, 롤백 시간 95% 감소 (30분 → 1~2분) 달성
- Jenkins 기반 Unity Android/iOS 빌드 자동화 및 Slack 연동 배포 알림 구축

모니터링 및 오피버빌리티 시스템 구축

장애 사전 감지 및 신속한 대응 체계 수립

- PLG Stack (Prometheus, Loki, Grafana) 기반 메트릭·로그 통합 모니터링 구축
- ELK → EFK Stack 전환 후 ECK Operator 기반 Elastic Stack 9.0 CRD 선언형 관리로 재설계 (TLS 자동 회전, 롤링 업그레이드 자동화)
- MTTR(평균 장애 복구 시간) 60~70% 개선, SLI/SLO 기반 서비스 가용성 99.9% 달성

자동화 및 효율화

Python, Bash를 활용한 운영 업무 자동화 및 개발 생산성 도구 구축

- 데이터 수집, 배포 프로세스, 문서 관리 자동화로 수동 작업 90% 이상 감소
- 내부 LLM 서비스(Ollama + Open WebUI) 구축으로 사내 보안 가이드라인 100% 준수 및 개발팀 생산성 향상
- GitLab Webhook 기반 문서 자동 동기화, Slack Bot 기반 APK 배포 알림 등 사내 도구 개발

Kubernetes 플랫폼 현대화 및 오픈소스 기여

네트워크·로깅 스택의 무중단 전환과 재사용 가능한 공용 차트 오픈소스 공개

- Ingress-nginx 11 인스턴스 → NGINX Gateway Fabric 단일 컨트롤플레인 + 11 Gateway CR 무중단 마이그레이션 (cutover 11/11, wildcard TLS 통합)
- ECK Operator 기반 로깅 스택을 로컬 차트 → OCI 차트로 무중단 전환 (CR/PVC 이름 보존, cluster_uuid 불변, 데이터 무손실)
- 오픈소스 Helm 차트 4종(nginx-gateway-cr, elasticsearch-eck, kibana-eck, certmanager-letsencrypt) 및 composite GitHub Actions 다수 공개, ArtifactHub / GitHub Marketplace 등록

데이터 엔지니어링 및 AI/ML 인프라

GCP 데이터 파이프라인부터 사내 LLM 서비스까지 직접 구축·운영

- BigQuery + Dataflow + Cloud Functions + Cloud Scheduler 조합으로 멀티 데이터 소스(MongoDB·CloudSQL·GA·Dune) → BigQuery → Google Sheets 자동 파이프라인 구축, 인프라 100% Terraform IaC
- 온프레미스 GPU 서버(RTX 3060) 위 Ollama + Open WebUI 기반 사내 LLM 서비스 Kubernetes 배포·운영, 외부 AI SaaS 구독 대체로 월 약 \$100 절감 및 사내 보안 가이드라인 100% 준수
- 데이터 수집 자동화 95% 달성, 휴먼 에러 제거로 데이터 누락률 0% 달성

주요 경력 (Professional Experience)

팬텀(콘크리트 스튜디오)

2024.10 ~ 현재

DevOps Engineer

게임 개발사의 DevOps 인프라 전담 엔지니어로, AWS 기반 클라우드와 온프레미스 서버를 설계·구축·운영하고 있습니다. 개발 서버와 테스트 서버는 온프레미스에, QA와 Production 서버는 AWS에 구축하여 하이브리드 아키텍처를 운영 중입니다.

현재 서비스 중인 게임 '소울즈'에 대해 퍼블리셔와 협업하여 2차 기술지원 및 운영을 담당하고 있으며, 개발 중인 신규 게임 프로젝트의 개발 생산성 향상을 위해 온프레미스·AWS 인프라를 구축하고 있습니다.

프로젝트 1: AWS-온프레미스 하이브리드 클라우드 아키텍처 구축

기여도 100% (단독 설계 및 구현) 2024.10 ~ 현재 (운영 중)

문제

전체 인프라를 AWS에서 운영하던 중 월 \$60,000의 높은 클라우드 비용이 발생하여 비용 최적화가 필요했습니다.

접근 전략

워크로드별 비용 분석을 수행한 결과, 개발·테스트 환경은 트래픽 변동이 적어 클라우드의 탄력성이 불필요했고, QA·Production 환경만 확장성과 안정성이 요구되는 상황이었습니다. 이에 개발·테스트 환경은 온프레미스로 전환하고, QA·Production 환경은 AWS에 유지하는 하이브리드 아키텍처를 설계했습니다. 양쪽 환경은 VPN 연결 없이 역할을 명확히 분리하여 독립 운영함으로써, 장애 전파를 원천 차단하는 구조를 채택했습니다. 양쪽 환경 모두 Terraform과 Ansible을 활용한 IaC 통합으로 운영 표준을 일원화(IaC화 100%)하여, 운영 복잡도 증가 없이 일관된 인프라 관리를 실현했습니다.

트러블슈팅

- EKS 환경에서 ALB Ingress 구성 시, 게임 클라이언트 버전별로 다른 백엔드로 라우팅해야 하는 요구사항 발생. ALB의 조건부 라우팅 규칙에서 커스텀 헤더 기반 라우팅과 Target Group을 조합하여 해결
- Kubernetes IPVS 모드에서 ClusterIP 서비스 간 통신 시 간헐적 타임아웃 발생. Cilium eBPF 기반 패킷 흐름 분석 및 IPVS conntrack 임계치 튜닝으로 해결하고, Cilium Network Policy 기반 zero-trust 보안 모델을 구축하여 파드 간 트래픽 가시성 및 보안 강화
- 온프레미스에서 빌드한 이미지와 AWS 측에서 별도로 push 된 이미지가 한쪽 registry에만 존재하여 cross-cluster 배포 시 외부 인터넷 경유 push/pull이 빈번해지고 빌드 시간·비용·실패율이 증가. Harbor의 native Replication (Harbor → ECR, event-based 정책)으로 한쪽 push 즉시 자동 복제 + 반대 방향은 ECR push 이벤트를 EventBridge → SNS → GitHub Actions workflow로 트리거하여 `skopeo copy`로 Harbor에 미러링, 양방향 동기화 자동화로 배포 시간 40% 단축
- 게임 서비스 rolling deploy 중 ALB target deregistration delay (기본 300초 → 운영 정책상 60초로 단축)와 Kubernetes Pod의 SIGTERM 처리 시점이 어긋나면서 일부 세션 connection drop 발생. (1) Pod의 lifecycle.preStop hook에 sleep 60s를 추가해 readiness probe가 먼저 fail하도록 유도하여 ALB가 target을 unhealthy로 인식 후 신규 요청 차단, (2) terminationGracePeriodSeconds를 ALB drain 60s + 진행 중 요청 처리 30s 합산 90s로 조정, (3) 게임 서버 컨테이너에 SIGTERM 핸들러를 구현해 진행 중인 매치 종료까지 대기하도록 처리하여 rolling deploy 중에도 game session connection drop 0건, zero-downtime 배포 달성

성과

- 워크로드 분석 기반 인프라 최적화로 월 비용 20% 절감 (\$60,000 → \$48,000), 연간 약 \$144,000 절감분을 신규 프로젝트 인프라에 재투자하여 리소스 선순환 구조 구축
- 개발 서버 온프레미스 이전으로 월 약 \$1,000 AWS 리소스 비용 추가 절감 및 리소스 사용량 최적화
- 환경별 역할 분리로 운영 안정성 확보 및 장애 전파 방지

기술 스택

AWS (EKS, EC2, ALB, Route53, ACM, EFS, Aurora MySQL, ElastiCache, CloudFront), Kubernetes, Terraform, Helm

프로젝트 2: 온프레미스 CI/CD 파이프라인 구축 및 고도화

기여도 100% (인프라 및 파이프라인 전체 담당) 2024.12 ~ 현재 (운영 중)

문제

개발자가 수동으로 빌드 후 서버에 배포하는 방식으로, 배포 시 평균 30분이 소요되었으며 휴먼 에러로 인한 장애가 빈번하게 발생했습니다.

접근 전략

GitLab CI와 ArgoCD를 조합한 GitOps 기반 자동 배포 환경을 구축했습니다. 개발자가 Git Push만 하면 자동으로 빌드→테스트→배포가 진행되며, ArgoCD가 Kubernetes 클러스터 상태를 모니터링하여 선언적 배포를 수행합니다. Kaniko를 활용한 Docker 이미지 빌드로 Docker-in-Docker 의존성을 제거하고, 멀티 환경(alpha/review/dev/staging) 배포를 단일 파이프라인에서 관리하도록 구성했습니다.

트러블슈팅

- ArgoCD Application의 syncPolicy.automated.selfHeal=true가 활성화된 환경에서 운영자가 일시 적용한 hot-fix를 컨트롤러가 즉시 되돌려 장애 디버깅이 어려워지는 문제 발생. prod 환경은 selfHeal=false + 수동 sync window로 분리하고 dev/staging 환경만 selfHeal=true를 유지, 모든 sync 이벤트를 Slack으로 알림 연동하여 변경 추적성과 안정성을 동시에 확보
- GitLab 업그레이드 후 GPG 키 만료로 CI Runner에서 패키지 설치 실패. GPG 키 갱신 프로세스를 자동화하여 재발 방지
- Kaniko 빌드 시 캐시 무효화로 빌드 시간이 급증하는 문제 발생. --cache=true --cache-ttl=24h --snapshot-mode=redo 옵션을 조합하여 캐시 적중률을 높이고 빌드 시간을 안정화

성과

- 배포 시간 67% 단축 (30분 → 10분)
- 수동 작업 자동화 90% 달성 (빌드, 배포, 설정 적용)
- 배포 자동화로 주간 배포 횟수를 3배 이상 증대 (1~2회 → 5~7회)시켜 신규 피처의 시장 출시 주기(Time to Market)를 단축하고, 배포 리소스 효율화로 개발팀이 본연의 기능 개발에 집중할 수 있는 환경 조성

기술 스택

GitLab CI/CD, ArgoCD, Jenkins, Kaniko, Kubernetes, Docker, Harbor

프로젝트 3: 중앙 집중식 로깅 시스템 구축 (ELK → EFK → ECK Operator 3단계 고도화)

기여도 100% (시스템 설계 및 구축) 2025.06 ~ 현재 (운영 및 고도화)

문제

온프레미스 환경에서 서버 및 컨테이너 로그가 분산되어 있어 장애 발생 시 원인 파악에 많은 시간이 소요되었으며, 통합 모니터링 체계도 없었습니다.

접근 전략

Phase 1 (ELK 구축): Elasticsearch, Logstash, Kibana, Filebeat를 활용한 중앙 집중식 로깅 시스템을 구축했습니다. 모든 서버와 컨테이너의 로그를 Filebeat로 수집하고, Logstash에서 JSON 파싱 및 필드 매핑을 처리한 뒤 Elasticsearch에 저장하여 Kibana 대시보드로 실시간 모니터링 및 검색이 가능하도록 했습니다.

Phase 2 (EFK 전환): Logstash의 높은 JVM 메모리 사용량(1GB+)과 Filebeat의 Helm 생태계 불일치, 로그 파이프라인의 커스텀 확장성 한계로 인해 Fluent Bit + Fluentd 기반 EFK Stack으로 전환했습니다. C 기반 경량 수집기인 Fluent Bit이 DaemonSet으로 각 노드에서 Kubernetes 메타데이터 자동 태깅과 함께 로그를 수집하고, Fluentd가 멀티 output을 지원하는 로그 가공/필터링을 처리한 뒤 Elasticsearch로 전달하는 구조로 리팩토링했습니다. 차트 업그레이드 자동화 스크립트를 개발하여 upstream Helm 차트 자동 업그레이드 시 커스텀 PVC 템플릿을 보존하도록 구성했습니다.

Phase 3 (ECK Operator 전환 + Stack 9.0 major upgrade): Elastic 공식 Helm Chart 업데이트가 2년 이상 중단되고 CRD 기반 선언형 관리(ECK)가 주류로 전환됨에 따라, Elastic Stack을 8.5.1에서 9.0.0으로 major upgrade하고 ECK Operator 3.3.2를 elastic-system 네임스페이스에 도입했습니다. Elasticsearch/Kibana를 Custom Resource로 재정 의하고 로컬 차트(CR wrapper) 형태로 관리하여 repo convention과 정렬했으며, 병렬 배포 (monitoring → logging 네임스페이스) 후 cutover 전략을 채택하여 무중단 전환을 수행했습니다. ECK 기반 체계에서는 TLS 인증서 자동 발급/회전(1년 / 30일), elastic 계정 패스워드 Helm template 관리, CR spec.version 한 줄 변경으로 완료되는 롤링 업그레이드 체계를 확립하여 수동 운영 부담을 제거했습니다.

트러블슈팅

- 게임 서버 로그에 포함된 null 바이트(\x00)로 인해 Logstash 파싱이 실패하는 문제 발생. mutate 필터에서 gsub으로 null 바이트를 사전 제거하는 전처리 단계를 추가하여 해결
- Elasticsearch 인덱스에서 동일 필드명에 서로 다른 데이터 타입이 들어와 필드 매핑 충돌(mapper_parsing_exception) 발생. 인덱스 템플릿에서 명시적 매핑을 정의하고, Logstash에서 타입 변환 필터를 적용하여 해결
- 로그 양 급증 시 Elasticsearch 디스크 워터마크 초과로 인덱스가 read-only 모드로 전환. ILM(Index Lifecycle Management) 정책을 적용하여 7일 초과 인덱스를 자동 삭제하고 디스크 사용량을 안정화
- [EFK] Fluent Bit이 NFS 볼륨에서 WAL(Write-Ahead Log) 파일 쓰기 실패. upstream Helm 차트의 _pod.tpl에 커스텀 PVC 볼륨 패치를 주입하고, 업그레이드 스크립트에서 자동 보존하도록 구성하여 해결
- [EFK] Fluentd output buffer overflow로 로그 유실 발생. chunk_limit_size와 flush_interval을 튜닝하고 overflow_action을 block으로 설정하여 로그 유실을 방지
- [ECK] Kibana 9부터 server.ssl.enabled=true에서 세션 쿠키에 Secure 플래그가 강제되어 HTTP 환경에서 브라우저가 쿠키를 누락해 로그인 불가. tls.disabled + publicBaseUrl http:// + xpack.security.secureCookies:false 조합으로 HTTP 로그인 경로를 복구
- [ECK] 3.3.x의 stackconfigpolicy-controller가 managedNamespaces 범위를 벗어난 리소스를 17분 간격으로 GET 시도하며 reconcile error 로그를 반복 생성. managedNamespaces: [(cluster-wide watch)로 후회하여 해소. RBAC는 이미 cluster-scoped(createClusterScopedResources:true) 여서 권한 델타 없음
- [ECK] Fluentd 공식 이미지에 ES9 variant가 없어 ES8 variant(v1.19.2-debian-elasticsearch8-1.4)를 사용할 수밖에 없는 상황. fluent-plugin-elasticsearch 5.4.4의 suppress_type_name:true로 ES 9의 _type 제거와 호환성을 확보하고, NFS PVC에 active marker 로그를 주입해 fluent-bit → fluentd → ES 전 경로를 엔드투엔드 검증

성과

- 로그 검색 시간 90% 단축 (개별 Pod 접속 확인 → Kibana 단일 인터페이스)
- 통합 로깅 시스템 구축으로 MTTR(평균 장애 복구 시간)을 70% 개선하여 서비스 가용성 99.9%를 상시 유지하고, 장애로 인한 잠재적 매출 손실 리스크를 최소화
- 일 평균 1GB의 로그 데이터를 실시간 처리 및 90일간 보관
- EFK 전환으로 로그 수집기 메모리 사용량 70% 감소 (Logstash JVM 1GB → Fluent Bit 50MB)
- Helm 기반 로깅 파이프라인 관리 통일 및 차트 업그레이드 자동화
- ECK Operator 전환으로 TLS 인증서·계정 패스워드·StatefulSet 업그레이드의 수동 운영 부담 제거. CR spec.version 한 줄 변경만으로 롤링 업그레이드 수행 가능
- Elastic Stack 8.5.1 → 9.0.0 major upgrade 완료 (2년간 누적된 업데이트 격차 해소)
- ECK Operator의 secureMode + ServiceMonitor로 controller-runtime 기반 메트릭을 kube-prometheus-stack에 자동 등록, Operator 내부 상태까지 관측 가능한 체계 확립

기술 스택

Elasticsearch, Fluent Bit, Fluentd, Logstash, Kibana, Filebeat, ECK Operator, CRD, Custom Resource, Kubernetes, Helm

프로젝트 4: 개발 생산성 도구 구축 (APK 배포 봇 · 문서 자동화 · LLM 서비스 · Git 미러링 · 정적 파일 서버)

기여도 100% (전체 시스템 설계 및 개발) 2025.02 ~ 현재

개발팀의 반복적인 수동 작업을 자동화하고 생산성을 높이기 위해, 5가지 사내 도구를 설계·개발·운영했습니다.

4-1. Slack APK 배포 자동화 봇 (4주, 2025.02)

문제

APK 빌드 완료 후 QA팀 전달이 수동으로 이루어져 공유 지연 및 버전 혼동 발생

해결

Jenkins APK 빌드 완료 시 Slack에 빌드 완료 메시지와 다운로드용 QR 코드를 자동 전송하는 Python 봇 개발, Kubernetes에 배포

성과

QA팀 전달 시간 90% 단축, QR 코드 기반 즉시 설치로 버전 혼동 제거, 빌드 이력 자동 기록

기술 스택

Python, Slack Bot API, Jenkins, Kubernetes, Docker

4-2. GitLab-Google Drive 문서 자동화 시스템 (2주, 2025.08)

문제

기술 문서를 GitLab에 마크다운으로 작성한 뒤 Google Drive에 수동 업로드하는 이중 작업이 발생

해결

GitLab Webhook과 Google Drive API를 연동하여 Push 시 자동 동기화. Python으로 마크다운→Google Docs 변환 후 업로드

성과

문서 관리 시간 80% 감소 (문서당 10분 → 2분), GitLab을 Single Source of Truth로 일원화

기술 스택

GitLab Webhook, Google Drive API, Python, Bash Script

4-3. 내부 LLM 서비스 구축 (4주, 2026.01)

문제

개발팀의 LLM 활용 수요 증가, 외부 SaaS 사용 시 비용 부담 및 사내 데이터 보안 우려

해결

온프레미스 GPU 서버(RTX 3060)에 Ollama와 Open WebUI를 Kubernetes 위에 배포. NFS 스토리지로 모델 저장소 구성

성과

개발팀 15명 대상으로 일 평균 100건 쿼리를 처리하여 코드 리뷰·문서 요약·디버깅 보조 도구로 활용. 외부 AI SaaS 구독 대체로 월 약 \$100 절감. 사내 보안 가이드라인 100% 준수 및 외부 데이터 유출 리스크 제거

기술 스택

Ollama, Open WebUI, Kubernetes, NFS, GPU (RTX 3060)

4-4. Git 저장소 미러링 도구 개발 (4주, 2026.02)

문제

AWS CodeCommit, GitLab, GitHub 등 멀티 플랫폼 간 소스 코드 동기화가 수동으로 이루어져 누락 및 지연 발생

해결

Go 기반 양방향 Git 미러링 도구(git-bridge)를 개발. SQS 폴링(CodeCommit) + HTTP Webhook(GitLab/GitHub) 이벤트 소스를 지원하며, 증분 동기화(git fetch)와 루프 감지로 안정적 운영.

유닛 테스트 및 GitHub Actions CI/CD 파이프라인을 구축하여 코드 품질을 확보하고, Kubernetes에 배포하여 Slack 알림 연동

성과

저장소 10개에 대해 일 평균 30건 동기화 이벤트를 처리하며 멀티 플랫폼 간 소스 동기화 100% 자동화, 수동 미러링 작업 제거. 오픈소스로 공개하여 GitHub Actions(multi-git-mirror)와 함께 활용

기술 스택

Go, AWS SQS, GitLab Webhook, GitHub Webhook, Kubernetes, Docker

4-5. 정적 파일 서버 자체 개발 (오픈소스, 2026.03 ~ 현재)

문제

기존에 사용하던 halverneus/static-file-server:v1.8.11은 단순 파일 서빙만 제공하여 디렉토리 리스팅 UI, 파일 프리뷰, 검색/필터, 일괄 다운로드 등 실제 운영에서 필요한 기능이 없었습니다. QA팀 APK 배포와 사내 파일 공유 환경의 사용성 개선 요구가 지속되었고, 관련 업스트림 차트의 유지보수도 정체된 상태였습니다.

해결

Go 기반 경량 정적 파일 서버(static-file-server)를 직접 설계·개발하고, GitHub Pages에 Helm repository(<https://somaz94.github.io/static-file-server/helm-repo>)를 운영하여 배포 표준을 확립했습니다.

Kubernetes에는 자체 Helm chart로 배포했으며, NFS StorageClass(nfs-client-nopath, ReclaimPolicy Retain, 5Gi)와 연결하여 장기 보존이 필요한 아티팩트를 안정적으로 호스팅합니다. 기존 halverneus/static-file-server:v1.8.11 기반 배포는 `_deprecated/static-file-server/`로 이관하여 완전 대체했습니다.

주요 기능

- 다크모드 지원 디렉토리 리스팅 UI (그리드/리스트 뷰 전환, 키보드 네비게이션)
- 파일 프리뷰 (이미지 / 비디오 / 오디오 / PDF / 텍스트 · 코드)
- 검색 · 필터 + URL 해시 공유, 다중 선택 + ZIP 일괄 다운로드
- Gzip 압축, Prometheus 메트릭 노출, JSON 구조화 로깅, Health Check
- APK 파일 Content-Type 자동 설정 (ingress annotation 기반)

성과

Apache-2.0 라이선스로 오픈소스 공개(Docker Hub · GitHub Pages Helm repo 운영). 일 평균 30건 다운로드 및 주 5회(매일 빌드) APK 배포 트래픽을 처리하며 APK 배포·문서 공유·빌드 아티팩트 배포 등 사내의 여러 파일 공유 요구를 단일 도구로 수렴했고, 자체 Helm chart 기반으로 배포·업그레이드·롤백을 표준화했습니다.

업스트림 의존성 제거로 보안 패치 및 기능 추가를 자체 개발 주기로 관리 가능.

기술 스택

Go, Helm Chart, GitHub Pages, Docker Hub, Kubernetes, NFS, Prometheus

프로젝트 5: Kubernetes 클러스터 운영 고도화

기여도 100% (모니터링 설계, 업그레이드 자동화, Helm 관리 프레임워크, NGF 마이그레이션, 오픈소스 Helm 차트 2종 공개) 2025.12 ~ 현재

클러스터 가시성, 운영 안정성, 네트워크 현대화를 목표로 모니터링 체계를 고도화하고 K8s 업그레이드·Helm 관리를 자동화했으며, 네트워크 컨트롤러를 Gateway API 기반으로 마이그레이션했습니다. NGF 마이그레이션 과정에서 도출된 CR 관리 구조와 ECK Operator 기반 로깅 스택을 각각 오픈소스 Helm 차트로 공개했습니다.

5-1. 모니터링 & 알림 고도화 (2026.01 ~ 현재)

문제

kube-prometheus-stack 기본 설치 상태로는 인프라·DB·네트워크 계층의 커스텀 알림·대시보드가 없었고, 물리서버와 VM 메트릭 수집 및 Cilium CNI 관측도 누락되어 장애 사전 감지가 어려웠습니다.

해결

kube-prometheus-stack을 기반으로 18개 커스텀 알림 규칙(Node 9개, Pod 2개, Cilium 4개)을 설계하고 10개 커스텀 Grafana 대시보드를 제작했습니다. MySQL·Redis·Elasticsearch·PostgreSQL exporter를 통합하고, 물리서버 4대와 VM에 Ansible로 node-exporter를 자동 배포했습니다. Cilium CNI의 agent/operator 메트릭을 kubernetes_sd_configs로 수집하고, Alertmanager에서 severity 기반 Slack 라우팅과 inhibit rules를 구성해 알림 품질을 확보했습니다.

성과

18개 커스텀 알림 규칙과 10개 대시보드로 인프라·DB·네트워크 전 계층 장애 사전 감지 체계를 구축하고, 물리서버·VM까지 포함한 통합 관측 커버리지를 확보했습니다. severity 기반 라우팅과 inhibit rules로 중복 알림을 제거하여 온콜 대응 품질을 개선했습니다.

기술 스택

Prometheus, Grafana, Alertmanager, Cilium, node-exporter, Ansible, Slack API

5-2. K8s 업그레이드 자동화 & Helm 차트 관리 프레임워크 (2025.12 ~ 2026.04)

문제

Kubespary 기반 K8s 업그레이드가 수동으로 진행되어 사전 검증·백업·호환성 확인 단계가 일관되지 못했고, 인프라 전체 18개 Helm 차트도 버전 관리 표준이 없어 업그레이드 스크립트가 차트별로 drift되었습니다. K8s 인증서도 6개월 만료 주기로 API Server 장애 리스크가 있었습니다.

해결

Kubespary 버전 관리 자동화 스크립트(사전 검증·인벤토리 백업·버전 호환성 확인·breaking change 감지)와 업그레이드 후 9단계 헬스체크 스크립트(노드·Pod·DNS·인증서·etcd·API Server)를 구축했습니다.

인프라 전체 18개 Helm 차트(external 14개 + local 4개)에 대해 업그레이드 스크립트 프레임워크를 구축하고, 4가지 캐노니컬 템플릿을 단일 source of truth로 관리하는 동기화 도구(check 모드로 CI drift 감지, apply 모드로 일괄 전파)를 함께 개발했습니다.

K8s 인증서 자동 갱신 스크립트는 확인 → 백업 → 갱신 → kubelet·apiserver 재시작 → kubeconfig 업데이트 → 최종 검증의 단계별 플로우와 롤백을 지원하며, 노드 10대에 crontab으로 6개월마다 새벽 3시에 주기 실행됩니다.

성과

9단계 자동 헬스체크로 K8s 업그레이드 후 검증 시간을 90% 단축하고, 18개 Helm 차트 업그레이드 프레임워크 + 캐노니컬 템플릿 동기화 도구로 인프라 버전 관리를 표준화하여 스크립트 본문 drift를 CI에서 자동 감지할 수 있게 되었습니다. 인증서 자동 갱신 스크립트와 crontab 주기 실행으로 노드 10대의 인증서 만료에 따른 API Server 장애를 사전에 제거했습니다.

기술 스택

Kubespray, Ansible, Helm, etcd, Shell Script, GitLab CI

5-3. Ingress-nginx → NGINX Gateway Fabric(NGF) 마이그레이션 (2026.04)

문제

온프레미스 클러스터에서 ingressClassName 기반 멀티테넌시를 위해 ingress-nginx 컨트롤러 11개 인스턴스(기본 1개 + public-a~j 10개, MetalLB LoadBalancer IP 10.10.10.55-58, 62-68 고정)를 운영하며 Deployment·Service·CRD·RBAC가 11세트로 중복되어 있었습니다. Kubernetes Gateway API가 v1.2 GA로 전환되며 ingress-nginx 프로젝트는 유지보수 모드로 전환될 예정이라, 네트워크 컨트롤러를 Gateway API 기반 후계자로 이관해야 했습니다.

해결

공식 후계자 NGINX Gateway Fabric(NGF) 2.x를 도입해 '단일 컨트롤플레인 + 11 Gateway CR' 구조로 단순화하고, MetalLB IP를 그대로 유지하여 DNS·방화벽 무변경 병렬 배포 + 점진 cutover 방식을 채택했습니다. Phase 0~7+ 7단계(MetalLB 풀 확장 → NGF 설치 → 관측 스택 5종 → HTTP-only / ApplicationSet 9개 → HTTPS 강제 앱(Harbor·Vaultwarden) → IP swap cutover 11/11 → Ingress 정리)를 2일 내 완료했습니다. ingress-nginx 어노테이션 10종을 Gateway API + NGF CRD(HTTPRoute filters, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy)로 1:1 매핑하고, HTTPRoute 차트 values 스키마를 4개 차트에 통일해 ApplicationSet 9개 일괄 전환에 재사용했습니다. self-signed wildcard cert 1장(*.concrit.us, 10년)으로 앱별 TLS Secret 2장(harbor-tls, vaultwarden-tls)을 통합했습니다.

트러블슈팅

- externalTrafficPolicy: Cluster + loadBalancerIP가 지정된 상태에서 Pod을 0으로 스케일해도 MetalLB가 IP를 해제하지 않아 NGF로의 IP swap이 불가능한 이슈 발생. Service를 ClusterIP로 patch하고 loadBalancerIP 필드를 제거하는 단계를 cutover 스크립트(cutover.sh)에 추가하여 강제 반환
- ingress-nginx admission ValidatingWebhookConfiguration이 잔존하면 endpoint Service가 Pod 0으로 응답 불가 → 해당 클래스의 Ingress UPDATE/DELETE가 2분 timeout으로 hang되는 이슈. cutover 단계에 VWC 선삭제를 추가하여 foregroundDeletion finalizer stuck도 자동 회피
- NGF chart 기본값 externalTrafficPolicy: Local로 인해 내부 Pod → 퍼블릭 도메인 → LB IP 호출 경로가 2분 timeout 발생(dataplane replica 1개라 노드 local miss). 11개 NginxProxy CR에 externalTrafficPolicy: Cluster를 명시하여 기존 ingress-nginx와 동일 정책으로 복원, CI generator 스테이지 회귀 해결

성과

11 컨트롤러 인스턴스 → 단일 NGF 컨트롤플레인 + 11 Gateway CR로 축소해 CRD·RBAC·Deployment 중복을 제거하고, 클래스 11개 cutover 11/11을 무사고로 완료했습니다(클래스당 실 다운타임 30~60초). HTTPRoute 차트 values 스키마 통일로 ApplicationSet 9개 일괄 전환 절차를 표준화하고, self-signed cert 2장을 wildcard 1장(10년)으로 통합해 수동 갱신 부담을 50% 감소시켰으며, Phase 0~7+ 전 단계를 2일 내 완료해 마이그레이션 기간을 최소화했습니다.

기술 스택

NGINX Gateway Fabric, Gateway API, HTTPRoute, ClientSettingsPolicy, ProxySettingsPolicy, RateLimitPolicy, BackendTLSPolicy, Helmfile, MetalLB, ArgoCD

5-4. nginx-gateway-cr 오픈소스 Helm 차트 공개 (2026.04)

문제

NGF upstream 차트는 컨트롤러와 CRD만 설치하며, 실제 트래픽 경로에 필요한 테넌트 레벨 리소스 (Gateway·NginxProxy·ReferenceGrant·ServiceMonitor·PodMonitor)는 제공하지 않습니다. NGF 2.x는 컨트롤러 /metrics가 Service가 아닌 Pod에 노출되어 전통 ServiceMonitor로는 수집이 부적합했고, 동일한 구조를 재사용할 수 있는 공용 차트도 없었습니다.

해결

NGF 마이그레이션 과정에서 추출한 local CR chart 구조를 다듬어, 5개 테넌트 레벨 리소스를 배포하는 범용 Helm 차트 nginx-gateway-cr을 오픈소스로 공개했습니다.

multi-Gateway 친화적 스키마(gateways[] 배열 + per-gateway 오버라이드), NGF 2.x 권장 PodMonitor 템플릿(controller + dataplane), values.schema.json 기반 입력 검증을 포함하며, OCI 레지스트리와 전통 Helm 레포 양쪽으로 배포해 설치 경로를 이원화했습니다.

성과

Apache-2.0 라이선스로 ArtifactHub networking 카테고리 등록되어, 유사 NGF 환경에서 재사용 가능한 공용 차트 산출물을 확보했습니다. 사내 NGF 마이그레이션 결과물을 그대로 재활용하여 자체 인프라에서는 helmfile 파이프라인으로 관리되고, 외부에서는 오픈소스로 배포되는 이중 활용 구조를 구축했습니다.

기술 스택

Helm Chart, Gateway API, NGINX Gateway Fabric, Prometheus Operator, OCI Registry, ArtifactHub

5-5. ECK Operator 기반 Elasticsearch / Kibana Helm 차트 공개 (2026.04)

문제

ECK Operator 도입 이후에도 Elasticsearch/Kibana 구성이 내부 storageClass·CR 이름 등 환경 의존이 많은 로컬 Helm 차트로만 관리되어 외부 재사용이 어려웠습니다. ECK Operator 기반 리소스를 한 차트로 묶어 공개하면서, 이미 로컬 차트로 운영 중인 기존 클러스터를 데이터 손실 없이 공개 차트로 전환할 경로도 함께 필요했습니다.

해결

charts/elasticsearch-eck(ES 템플릿 11종) + charts/kibana-eck(Kibana 템플릿 10종) 두 차트를 신규 설계해 CR·Secret·HTTPRoute·Ingress·BackendTLSPolicy·ServiceMonitor·NetworkPolicy 등 Elasticsearch/Kibana 운영에 필요한 리소스를 단일 values 로 배포하도록 구성했습니다.
values.schema.json(draft-07)으로 HA 옵션·스토리지·자원 입력을 검증하고 README에 Minimal/HA 프리셋과 환경별 storageClass 매핑을 명시했으며, OCI(ghcr.io/somaz94/charts)-gh-pages Helm 레포·ArtifactHub(monitoring-logging 카테고리, Apache-2.0) 세 경로로 동시 배포했습니다. OCI 버전 추적을 위해 scripts/upgrade-sync/templates/oci-chart-version.sh canonical 템플릿을 함께 작성해 사내 버전 추적 절차도 정비했습니다.

성과

mgmt 클러스터의 로컬 차트 → OCI 차트(elasticsearch-eck / kibana-eck 0.1.1) 전환 시 CR·PVC·Secret 이름을 전부 보존해 cluster_uuid 불변, Elasticsearch pod 재시작 0회·Kibana rolling restart 1회로 데이터 무손실 전환을 완료했습니다.
HA 롤링 kind 검증 3/3 PASS, 실 클러스터 설치에 Elasticsearch 71초·Kibana 57초 만에 Ready/green 에 도달했으며, ArtifactHub 공개로 nginx-gateway-cr 에 이어 두 번째 공개 차트를 확보해 사내 helmfile 파이프라인과 외부 오픈소스 배포의 이중 활용 구조를 로깅 스택까지 확장했습니다.

기술 스택

Helm Chart, ECK Operator, Elasticsearch, Kibana, Gateway API, OCI Registry, ArtifactHub, Prometheus Operator, Kubernetes

프로젝트 6: 인프라 보안 체계 구축

기여도 100% (설계, 배포, SSO 연동) 2026.04 ~ 현재

인프라 운영에 필요한 민감 정보를 안전하게 관리하기 위한 보안 체계를 구축하고 있습니다.

6-1. Vaultwarden 비밀번호 관리 시스템 (2026.04)

Vaultwarden(Bitwarden 호환 서버)을 Kubernetes에 Helm 차트로 배포하고, GitLab SSO(OpenID Connect)를 연동하여 기존 계정으로 바로 사용할 수 있도록 구성했습니다. Self-signed TLS 인증서를 extraObjects로 Helm에서 관리하고, 조직/컬렉션 기반 팀별 접근 제어를 설정했습니다.
자동 백업 CronJob(날짜별 SQLite 덤프, 30일 retention)과 대화형 restore 스크립트를 구축하여 데이터 안정성을 확보했습니다.

성과

- 사용자 70명 · 시크릿 50건을 중앙 관리 체계로 전환하여 보안 리스크 해소 및 인수인계 효율화
- GitLab SSO 연동으로 기존 사내 계정으로 즉시 로그인 가능, 별도 계정 발급/삭제 관리가 불필요해져 계정 관리 효율성 향상

기술 스택

Vaultwarden, Helm, OpenID Connect, GitLab SSO, Kubernetes

너디스타

2023.03 ~ 2024.07

DevOps Engineer

게임 및 블록체인 기반 스타트업에서 DevOps 엔지니어로 근무하며, AWS에서 GCP로의 대규모 클라우드 마이그레이션 프로젝트를 총괄했습니다.
GCP Startup Program을 활용한 비용 최적화와 GCP의 글로벌 네트워크 성능을 활용하기 위해 전환을 진행했으며, 전체 서비스 마이그레이션을 성공적으로 완료했습니다. 온프레미스 환경은 GitLab, Production 환경은 GitHub로 소스를 관리하는 프로젝트별 이원화 체계를 운영했습니다.

프로젝트 1: AWS → GCP 대규모 클라우드 마이그레이션 및 CI/CD 재구축

기여도: 인프라 아키텍처 설계·마이그레이션 전략 수립 및 실행 총괄 70%, CI/CD 파이프라인 구축 100% 7개월 (2023.05 ~ 2023.12)

문제

AWS 비용이 지속적으로 상승하고 있었으며(월 \$10,000+), 특히 NAT Gateway와 데이터 전송 비용이 높았습니다. 또한 멀티 리전 게임 서비스를 위해 GCP의 글로벌 로드밸런싱이 필요했으며, 기존 Jenkins 기반 배포 시스템은 설정 관리가 복잡하고 배포 이력 추적이 불가능했습니다.

접근 전략

3단계 마이그레이션 전략을 수립하여 순차적으로 진행했습니다. 1단계로 개발 환경을 GCP에 먼저 구축하여 검증하고, 2단계로 QA 환경을 이전한 뒤, 3단계로 프로덕션 환경을 이전했습니다.

네트워크 설계는 Shared VPC (Host Project / Service Project 분리) 기반으로 구성하고, GitHub Actions 의 GCP 인증은 Workload Identity Federation 으로 전환하여 서비스 계정 키 발급/순환 부담을 제거하고 OIDC 기반 단명 토큰으로 대체했습니다. DNS 는 서브도메인 위임 (Hosting.kr → AWS Route53 → GCP Cloud DNS) 으로 사전 검증 단계를 거친 뒤, 최종 NS 변경 시점에만 점검 공고로 무중단 cutover 를 수행했습니다. Filestore 의 SSD 최소 2.5TB

비용 문제는 Compute Engine + pd-balanced 1TB 디스크 기반 NFS 서버 자체 구축으로 회피하여 운영 비용을 최소화했습니다. Cloud Armor 로 region 차단 + IP 화이트리스트 WAF 정책을 구성하고, Cloud CDN 의 URL Map 을 HTTP / HTTPS 두 개로 분리하여 HTTP → HTTPS 강제 리다이렉트와 정적 자산 캐싱을 함께 활성화했습니다.

Terraform 으로 GCP 인프라를 IaC 화 (IAM 제외 100%) 하고, 마이그레이션 완료 후 GitLab CI 와 ArgoCD 를 조합한 GitOps 기반 CI/CD 파이프라인을 재구축했습니다. Helm Chart 로 환경별 설정을 표준화하고 Git 커밋 기반 배포 이력 관리 체계를 확립했습니다.

트러블슈팅

- AWS ALB 기반 라우팅을 GCP 글로벌 로드밸런서로 전환하는 과정에서 헬스체크 방식과 백엔드 서비스 구성 차이로 트래픽 라우팅 이슈 발생. GCP NEG(Network Endpoint Group) 구조를 분석하여 GKE 워크로드에 맞는 설정으로 재구성
- AWS RDS에서 Cloud SQL로 데이터 마이그레이션 시 캐릭터셋 차이로 인한 데이터 깨짐 발생. 사전 검증 스크립트를 작성하고 단계별 데이터 정합성 체크 프로세스를 도입하여 해결
- AWS와 GCP의 서브넷 모델 차이(AZ 기반 vs 리전 기반)로 인한 네트워크 설계 이슈. CIDR 블록을 재설계하고 방화벽 규칙을 GCP 태그 기반으로 전환
- GKE Ingress + BackendConfig 의 healthcheck requestPath 응답이 누락되어 503 에러 발생. 해당 경로 응답을 보장하고 ManagedCertificate / FrontendConfig 와 정렬하여 외부 진입 트래픽 정상화
- Cloud CDN 마이그레이션 후 HTTP → HTTPS 리다이렉트 누락으로 게임 클라이언트 파일 다운로드가 실패. URL Map 을 HTTP / HTTPS 두 개로 분리 (default_url_redirect.https_redirect=true) 하고 HTTP forwarding rule 에 80 포트 target HTTP proxy 를 별도 매핑하여 해결

성과

- 클라우드 비용 50% 절감 (월 \$10,000 → \$5,000), 연간 약 \$60,000 비용 절약
- 3단계 전략으로 전체 서비스 마이그레이션 완료. 리소스 복사 방식으로 다운타임 최소화하고, 최종 DNS 전환 시에는 게임 서비스 특성을 반영해 약 2시간 내외 점검 공지 후 전환 수행
- 전체 GCP 인프라를 Terraform으로 IaC 관리 체계 구축 (TFLint/Checkov 정적 분석 및 Terratest 단위 테스트 도입으로 코드 품질 확보)
- 배포 시간 50% 단축 (40분 → 20분), 롤백 시간 95% 감소 (30분 → 1~2분)
- 주간 배포 횟수 3배 증가 (2회 → 6회), Git 커밋 기반 배포 이력 100% 추적 가능
- Workload Identity Federation 도입으로 GitHub Actions 의 GCP 서비스 계정 키 관리 부담을 제거하고 키 유출 리스크를 원천 차단, 단명 토큰 기반 보안 모델로 전환

기술 스택

GCP (GKE, Cloud SQL, Cloud Storage, VPC, Global LB 등), Shared VPC, Workload Identity Federation, Cloud Armor, Cloud CDN, Cloud DNS, NFS, Terraform, GitLab CI, ArgoCD, GitHub Actions, Kubernetes, Helm, Docker

프로젝트 2: 게임 데이터 수집 자동화 시스템 개발

기여도 100% (Python 스크립트 개발 및 운영) 3개월 (2024.01 ~ 2024.03)

문제

게임 및 블록체인 데이터를 수동으로 수집하고 있어 분석가가 매일 2~3시간을 데이터 수집에 소비했으며, 휴먼 에러로 인한 데이터 누락이 빈번하게 발생했습니다.

접근 전략

MongoDB · CloudSQL · Google Analytics · Dune 등 멀티 데이터 소스를 BigQuery 로 적재하고, 분석가용 Google Sheets 까지 자동 갱신되는 데이터 파이프라인을 GCP 서비스 조합으로 구축했습니다. MongoDB 는 Dataflow Flex Template 기반 ETL 로 BigQuery 에 적재하고, CloudSQL 은 BigQuery 의 직접 connection 기능을 활용하며, GA / Dune 은 각자의 API (Dune 은 별도 API Key) 로 호출하여 일관된 인덱스 스키마로 정규화했습니다.

Python Cloud Function 이 BigQuery 쿼리 결과를 Google Sheets API 로 적재하고, Cloud Scheduler 가 Daily (전일 데이터) / Monthly (전월 데이터) 두 가지 주기로 자동 트리거합니다. BigQuery 적재 후에는 별도 중복 제거 Cloud Function 이 정기 실행되어 데이터 정합성을 유지합니다. 전체 인프라 (BigQuery Dataset · Cloud Function · Scheduler · Cloud Storage · Artifact Registry · IAM 권한 · Service Account) 는 Terraform 으로 IaC 관리하여 환경 재구축과 변경 추적을 표준화했습니다.

성과

- 데이터 수집 자동화 95% 달성 (일 2~3시간 수동 작업 제거)
- 자동화로 휴먼 에러 제거, 데이터 누락률 0% 달성
- 전체 인프라 Terraform IaC 화로 동일 파이프라인을 다른 GCP 프로젝트에 30분 내 재구축 가능, 변경 이력 100% Git 추적

기술 스택

BigQuery, Dataflow, Cloud Functions, Cloud Scheduler, Cloud Storage, Artifact Registry, Terraform, Python, Google Sheets API, Dune API, MongoDB, CloudSQL, Docker

프로젝트 3: PLG Stack 기반 모니터링 시스템 구축

기여도 100% (시스템 설계 및 구축) 1개월 (2024.04)

문제

Kubernetes 클러스터와 애플리케이션의 실시간 모니터링 체계가 없어 장애 발생 시 사후 대응만 가능했으며, 원인 파악에 과도한 시간이 소요되었습니다.

접근 전략

Prometheus로 메트릭을, Loki로 로그를 수집하여 Grafana 대시보드로 통합 시각화하는 PLG Stack을 구축했습니다. CPU·메모리·네트워크 사용률과 애플리케이션 로그를 실시간으로 모니터링하고, 임계치 초과 시 Slack 알림을 자동 발송하도록 구성했습니다.

성과

- 임계치 기반 알림으로 장애 사전 감지율 80% 달성
- 실시간 모니터링 및 알림 체계로 장애 대응 시간 60% 감소
- 서비스 가용성 99.5% → 99.9% 향상
- SLI/SLO 수립 및 Alertmanager 알람 노이즈 최적화로 운영팀 피로도 감소 및 실질적 장애 감지 정확도 향상

기술 스택

Prometheus, Loki, Grafana, Promtail, Fluent-bit, Slack API

아이오차드2022.04 ~ 2023.03

Infra Engineer

PaaS(Kubernetes + OpenStack + Ceph) 기반 인프라 솔루션을 고객사에 구축하고 기술 지원을 제공하는 역할을 수행했습니다. 금융권 및 공공기관 고객을 대상으로 온프레미스 클라우드 인프라를 설계·구축하고, 고객사 운영팀 대상 교육을 진행했습니다.

프로젝트 1: 고객사 맞춤형 PaaS 솔루션 구축

기여도 100% (인프라 설계 및 Kubernetes 클러스터 구축 담당) 11개월 (2022.04 ~ 2023.03)

문제

고객사마다 요구하는 환경과 서버 스펙이 상이하여, 표준화된 솔루션으로는 대응이 어려웠습니다.

접근 전략

고객사 요구사항에 맞춰 Kubernetes 클러스터를 HA(High Availability) 구성으로 설계하고, OpenStack으로 가상머신 관리 환경을 구축했습니다. Ceph로 블록·파일·오브젝트 스토리지를 통합 제공하고, 고객사 운영팀에 인프라 운영 교육을 병행했습니다.

성과

- 5개 고객사에 PaaS 솔루션 성공적으로 구축 및 납품
- Kubernetes 클러스터 HA 구성으로 99.9% 가용성 달성
- Ansible 기반 서버 프로비저닝 및 구성 관리 자동화로 고객사별 배포 시간 단축 및 환경 불일치(Configuration Drift) 이슈 제로화

기술 스택

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

프로젝트 2: DP사 Kubernetes 운영 교육 프로그램 수행

기여도 100% (교육 설계 및 진행) 6개월 (2022.06 ~ 2022.11)

문제

DP사 운영팀이 Kubernetes·컨테이너 기반 인프라 운영 경험이 부족하여, PaaS 솔루션 도입 후 자체 운영에 어려움이 예상되었습니다.

접근 전략

Kubernetes 기초부터 클러스터 운영·트러블슈팅까지 단계별 교육 커리큘럼을 설계하고, 실습 환경을 구성하여 실무 중심의 교육을 진행했습니다. OpenStack·Ceph 스토리지 운영 교육도 병행하여 전체 PaaS 스택의 운영 역량을 확보하도록 지원했습니다.

성과

- 수강자 평균 10명·회당 교육 4시간 규모로 커리큘럼을 운영하여 DP사 운영팀의 Kubernetes 자체 운영 체계 확립
- 교육 완료 후 고객사 기술 문의 건수 감소, 자체 장애 대응 가능 수준 달성
- 교육 자료를 표준화하여 이후 타 고객사 교육에도 재활용

기술 스택

Kubernetes, OpenStack, Ceph, Ansible, Rocky Linux/CentOS

프로젝트 3: Kubernetes 인증서 자동 갱신 프로세스 개선

기여도 100% (단독 수행) 2주 (2022.11)

문제

Kubernetes 클러스터 인증서가 1년마다 만료되어 고객사마다 연간 약 5시간의 갱신 작업이 필요했으며, 만료 시 서비스 장애가 발생할 위험이 있었습니다.

접근 전략

Kubernetes 인증서 관리 프로세스를 분석하고, kubeadm 설정을 수정하여 인증서 유효기간을 1년에서 10년으로 연장했습니다. 기존 운영 중인 클러스터에도 적용 가능한 자동화 스크립트를 개발하여 전체 고객사에 배포했습니다.

성과

- 인증서 갱신 주기 10배 연장 (연 1회 → 10년에 1회)
- 고객사 운영 부담 연간 약 50시간 절감 (고객사 10곳 기준)
- 인증서 만료로 인한 장애 리스크 제거

기술 스택

Kubernetes, kubeadm, Bash Script, OpenSSL